1 Lesson 23-10-26

1.1 Interconnections and communation protocols

We are finally getting to connecting our components together. As components are usually quite smart themselves, simple interconnections are not fit for the purpose, and we have to define more complex *communication protocols*.

There are a number of protocols:

- **UART** (*Universal Asynchronous Receiver Transmitter*);
- SPI;
- I2C;
- **CAN** (*Controller Area Network*).

Interconnections can be at several levels:

- *Within* an integrated circuit:
  - Metal layers, parallel bits
  - Lots of wires, high performance
  - Low parasitics, low noise
  - Delays can be accurately estimated (Elmore)
- Between *integrated circuits* on the board
  - Copper lines on a PCB, parallel or serial
  - Higher noise, higher delays than in an integrated circuit
- Between *boards* in the same embedded system:
  - Cables made of wires, parallel or serial
  - Even higher noise and delays
- Between *embedded systems*:
  - Cable made of wires or wireless connections
  - Unpredictable delays and extremely high noise
  - Need to use at least error detection in many cases

The goal of interconnections is to transfer bits as quickly as possible, while not sacrificing metrics:

- Correctness (definitely, as we are transfering digital information);
- Reliability;
- Power consumption;
- Costs;
- ...

Interconnections are defined at different *levels*, as we usually see in the OSI model:

- **Physical**, which refers to the actual electrical wire interconnects.
- **Protocol**, which refers to how the signals these wires transmit are organized;
- **Application**, of interest from the computer engineering point of view, refers to the actual application messages our systems are sending to each other.

Physical and protocol focus on how to transfer bits from a transmitter to a receiver. This requires bit synchronization (especially for serial protocols). In this, we examine the information needed to successfully complete a transfer, with no details on the semantic content of the messages that are transferred.

1.1.1 Interconnection delays

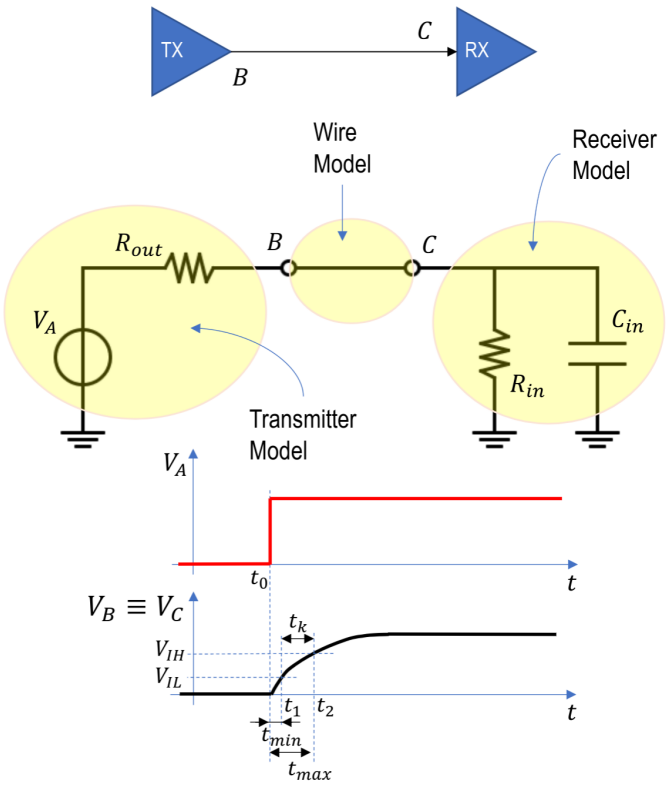
When connecting the output of a logic gate to the input of another logic gate, **delays** affect the exchange of data. We're interested in *measuring* these delays: even if the transmitter generates an ideal step, the waveform at the receiver is generally exponential with a certain time constant. This is, as usual, due to parasitic resistance, capacitance and inductance of transmitter, receiver and wire. The different output response results in a difference in switching thresholds of the receiving logic gates. This causes different delays for nominally identical circuits.

The delay is affected by an *uncertainty*, called **skew**.

Once the *skew* is known, the delay can be expressed as:

$$t_d = t_{\text{prop}} \pm \frac{t_k}{2}$$

Where t_{prop} is an average propagation delay.



The delay depends on the receiving gate threshold, between V_{IL} and V_{IH}.

We see that at the receiver we get an exponential ramp with time constant:

$$\tau = C_{in} \cdot (R_{in} \parallel R_{out})$$

Now, based on the input gate threshold, the delay time sits between t_{min} and t_{max}:

$$t_{\text{min}} = t_1 - t_0$$
$$t_{\text{max}} = t_2 - t_0$$

Where t_1 is where we meet V_{IL} and t_2 is where we meet V_{HL}.

The **skew** t_k is exactly:

$$t_k = t_{\text{max}} - t_{\text{min}} = t_2 - t_1$$

So we know that to correctly characterize the entire interconnection we have to:

- Find the minimum of all minimum delays t_{min};
- Find the maximum of all maximum delays t_{max};
- Compute the average t_{avg} and skew t_k.

$$t_{\text{avg}} = \frac{t_{\text{min}} + t_{\text{max}}}{2} = \frac{t_1 + t_2 - 2t_0}{2}$$
$$t_k = t_{\text{max}} - t_{\text{min}} = t_2 - t_1$$

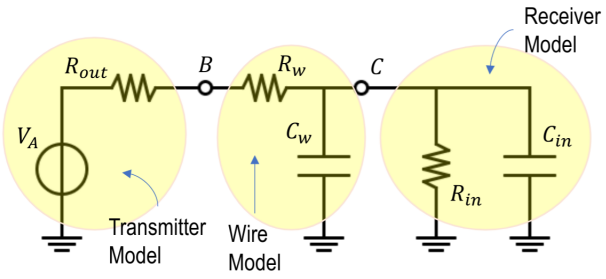
To summarize, we model an interconnection as lumped parasitic resistance R_w and capacitance C_w. The transmission shows a delay due to the input parasitic capacitance of the receiver, and due to the lumped parasitic capacitance of the wire.

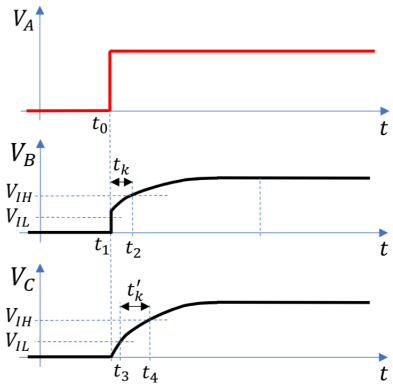
The delay shows an *uncertainty* (skew) due to different thresholds among nominally identical logic gates, and due to possibly different positions of the receiver along the interconnection wire. In short, wire length causes different parasitic resistance R_w and capacitance C_w values, which affects both the delay and its uncertainty (skew).

1.1.2 Models

Let's see some popular models for delays in interconnects:

- **Exponential**
  Let's complete the previous model by putting the *wire* back into the picture. As always, we model it with a parasitic capacitance and resistance:



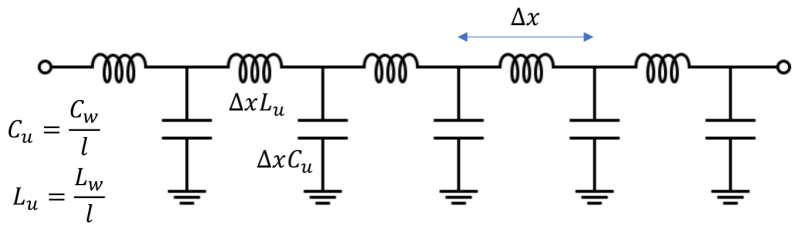


In this case our time constant changes:

$$\tau = (C_{\text{in}} + C_w) \cdot (R_{\text{in}} \parallel (R_{\text{out}} + R_w))$$

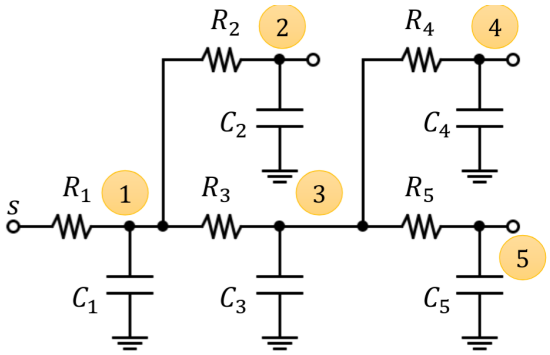
Note that this situation is made even more complex by the fact that the wire constants C_w and R_w *change* with the length of the wire, meaning that we have an added layer of uncertainty.

- **Transmission lines**
  The idea is to model an interconnection using distributed parasitics: we take a wire and divide its length l into N segments, each lumped. This means each segment has length $\Delta x = \frac{l}{N}$. We are interested in what happens if we take the limit of elements going to infinity ($N \to +\infty$).



  If considering only inductance and capacitance, this is the **ideal transmission line with no losses**. Signal propagates with a certain speed, with possible reflections, and the delay depends on the position of the receiver along the line. For this we can compute an average and a skew.
  Note that, if the resistances are also included, losses are introduced in the model and we get the **ideal transmission line with losses**.

- **Elmore delay**
  Note that this already appeared in 7.0.2, for NAND topology memory timings. There's a useful note here: `https://raw.githubusercontent.com/seggiani-luca/ees-notes/fcbb33c4bfdef8fc531aa81c04cddb69d309ad2a/notes/nand_topology_delay/main.pdf`.
  The idea once again is to model an interconnection using distributed parasitics, by dividing the wire into N segments, each with a lumped model. For complex interconnections, we can determine a tree stemming from a single node s, where each segment is characterized by resistance R_j, and capacitance C_j.



  Matematically, the time constant τ_i of the delay at node i is given by the sum of all RC segments up to i:

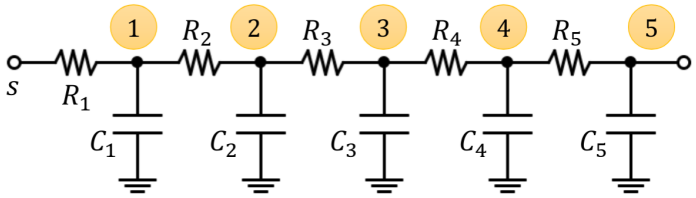
$$\tau_i = \sum_{k=1}^{N} C_j R_{ik}$$

  Where R_{ik} is the shared path resistance between node i and k:

$$R_{ik} = \sum R_j \text{ where } (R_j \in \text{path}(s,i) \cap \text{path}(s,k))$$

- **Elmore delay line**

Elmore delay is easily applied to an RC segment line:



The shared path resistance is always the sum of all resistance between root s and the considered node. For the last node i the shared path resistance with node k is always:

$$R_{ik} = R_{kk} = \sum_{j=0}^{k} R_j$$

At this point we have that, for a wire of length l divided into N identical segments:

- All resistances are identical: $R_i = r\frac{l}{N}$ (where r is the resistance per unit of length);
- All capacitances are identical: $C_i = c\frac{l}{N}$ (where c is the capacitance per unit of length).

This means we can apply Gauss's formula:

$$\tau_{\text{tot}} = \tau_N = R_i C_i \frac{N(N+1)}{2} = R_{\text{tot}} C_{\text{tot}} \frac{N+1}{2N}$$

In the limit this is:

$$\tau \to \frac{1}{2} N^2 R_i C_i$$

Where:

$$R_{\text{tot}} = N R_i$$
$$C_{\text{tot}} = N C_i$$

Which should be the case we saw in our notes.
There is as slight notation difference between this in the notes:

- In the notes we took each capacitor at contributing t_i to the total delay time, thus giving:

$$t_{\text{tot}} = \sum_i t_i$$

- Here we took t_i as the delay at node i, already containing the delay from previous nodes (see the formula above for τ_i), thus giving:

$$t_{\text{tot}} = t_N$$

Nevertheless, the result we get in both cases is the same, we just changed the definitions of the derivations.

1.1.3 Parallel vs serial

To transmit a word of n bits, bits are transmitted according to a synchronization signal. The synchronization signal is a clock, where:

- The clock frequency is f_{ck};
- The clock period is $t_{\text{ck}} = \frac{1}{f_{\text{ck}}}$.

Note that the clock shall be transmitted as well as the signal. There are several ways of communicating the clock. We wish to compare these techniques based on throughput:

- The number of bits per second is Thr_{bits};
- The number of bits per second is $\text{Thr}_{\text{words}}$.

Assume the transmitted symbols represent single bits: we can increase performance by using multiple bits per symbol, thus lowering noise margins.

- **Parallel**
  This is the idea of **parallel**, as opposed to **serial**, transmission. We transmit an entire word using n parallel wires, with an additional wire for the clock.
  This increases performance linearly:
  - The number of bits per second becomes $\text{Thr}_{\text{bits}} = nf_{\text{ck}}$;
  - The number of bits per seconds stays $\text{Thr}_{\text{word}} = f_{\text{ck}}$, assuming the n parallel lines form a complete word. Because of this, the number of bits is often a power of 2.

  Parallel is certainly performant, but this comes at a cost we need a lot of wires for large busses, and its not good for long interconnections as its heavy, bulky, and of large size;
- **Serial**
  Serial is parallel's chill cousin, where we usually transmit the word and the clock using a single wire. Bits are multiplexed in time, one per clock cycle, and one after the other, sequentially (LSB or MSB first). This inevitably introduces latency at the word level. A problem is also that the clock must usually be embedded in some way with the data, which needs special encodings for clock recovery.
  The effects on performance are:
  - The number of bits per second becomes $\text{Thr}_{\text{bits}} = f_{\text{ck}}$;
  - The number of bits per seconds becomes $\text{Thr}_{\text{word}} = \frac{f_{\text{ck}}}{n}$, assuming again that n is the number of bits in a complete word.

  With them being mostly opposite, serial is way cheaper and simpler than parallel, if not a bit less performant.

1.1.4 Clock

There are limitations on the clock frequency. The receiving end usually has flip-flops to store the incoming data: setup and hold times t_s and t_h must be satisfied at the receiving end.

Let's consider first a simplified model (no skew t_k), and then a more accurate one.

- **No skew**
  With no transmission delays and perfect synchronization, the maximum f_{ck} depends

on t_s and t_h of the receiving flip-flops only:

$$t_{\text{ck}} > t_s + t_h, \quad f_{\text{ck}} < \frac{1}{t_s + t_h}$$

This is fine as, if clock and data have different (but known) delays, the transmitter properly produces the clock earlier or later so that setup and hold times are satisfied at the receiver.

To satisfy these contraints we define 4 parameters:

– t_1^{TX}: time the data is stable at the TX side before the active clock edge;
– t_1^{RX}: time the data is stable at the RX side before the active clock edge;
– t_2^{TX}: time the data is stable at the TX side after the active clock edge;
– t_2^{RX}: time the data is stable at the RX side after the active clock edge.

It must be that:

$$t_1^{RX} \geq t_s, \quad t_2^{RX} \geq t_h$$

- **Skew**
  Let's reintroduce skew. This means the delay is not the constant t_{prop} (unknown but not significant) anymore, but a varying:

  $$t_d = t_{\text{prop}} \pm \frac{t_k}{2}$$

  Where t_k is our skew.
  With the phase delay between data and clock not maintained, delays are unpredictable and different in clock and data. Delays change from one bit transmission to the next, and the transmitter should consider the worst case scenario.
  For the setup time we have two sources of badness:

  – Clock arrives early at RX after:

  $$t_d^{ck} = t_{\text{prop}} - \frac{t_k}{2}$$

  – Data arrives late at RX after:

  $$t_d^{data} = t_{\text{prop}} + \frac{t_k}{2}$$

  $$t_1^{RX} = t_1^{TX} + t_d^{ck} - t_d^{data} = t_1^{TX} - t_k$$

  Since it should be that $t_1^{RX} > t_s$, it has to be that:

  $$t_1^{TX} > t_s + t_k$$

  Via similar hold time considerations (t_2) we get the frequency constraint:

  $$f_{\text{ck}} < \frac{1}{t_s + t_h + 2t_k}$$